# Core Concepts of DBMS, RDBMS, and SQL Commands

## What is DBMS?

A Database Management System (DBMS) is software that stores, manages, and retrieves data efficiently. It allows multiple users and applications to interact with the data securely and systematically.

## Limitations of File System → Why DBMS?

Before DBMS, data was stored in traditional file systems which had major limitations:

• Data Redundancy: Same data stored in multiple places.

• Lack of Security: No controlled access.

• Poor Data Integrity: Hard to maintain accuracy.

• No Concurrent Access: Multiple users could corrupt data.

• Difficult Backup & Recovery.

DBMS solved these issues by providing centralized management, security, consistency, and efficient data handling.

## What is RDBMS?

A Relational DBMS stores data in tables and maintains relationships using keys.

Simple Example:

A database with:

- Students Table
- Courses Table
- Enrollments Table
- All these tables will be connect to each other by establishing some relationship keys.

## Advantages of RDBMS over DBMS

• Structured data storage using tables.

• Eliminates redundancy through normalization.

• Uses keys to maintain relationships.

• Ensures ACID properties for reliability.

• Supports powerful SQL queries.

• Better security and access control.

## Key Components of RDBMS

• Entity Relationship Diagram

• Relationships

• Keys

• Constraints

• Normalization

• ACID Properties

## ER Diagram:

An Entity-Relationship Diagram visually represents entities (tables), their attributes (columns), and relationships (connections between tables). It is used during database design to understand structure before implementation.

## Relationships in RDBMS:

**a) One-to-One (1:1)**

One record in Table A relates to exactly one record in Table B.

Example:
A person has one passport, and each passport belongs to one person.

Tables:

- Person(person_id, name)
- Passport(passport_id, issue_date, person_id [FK])

**b) One-to-Many (1:N)**

One record in Table A relates to multiple records in Table B.

Example:
One teacher teaches many students.

Tables:

- Teacher(teacher_id, name)
- Student(student_id, name, teacher_id [FK])

**c) Many-to-Many (M:N)**

A record in Table A can relate to many in B and vice versa.
Requires a junction (bridge) table.

Example:
A student can enroll in multiple courses, and each course has many students.

Tables:

- Student
- Course
- Enrollment(student_id, course_id)

## Keys in RDBMS:

**a) Primary Key:**

Uniquely identifies each record in a table.
Cannot be NULL.

Example:
student_id in Student table.

**b) Foreign Key:**

A column that references a Primary Key in another table.

Example:
course_id in Enrollment table referencing Course table.

**c) Composite Key:**

A key made of multiple columns.

Example:
Enrollment table:
Primary Key = (student_id, course_id)

## Constraints:

Rules applied to maintain data integrity.

Common Constraints with Examples:

- **NOT NULL:** Column cannot be empty

  name VARCHAR(50) NOT NULL

- **UNIQUE:** No duplicate values

  email VARCHAR(50) UNIQUE

- **PRIMARY KEY:** Unique + Not Null

  id INT PRIMARY KEY

- **FOREIGN KEY:** Link between tables

  FOREIGN KEY (dept_id) REFERENCES Department(id)

- **CHECK:** Enforces conditions

  CHECK (age >= 18)

- **DEFAULT:** Assigns a default value

  status VARCHAR(10) DEFAULT 'active'

## Normalization:

A process to organize data to reduce redundancy.

Normal Forms:

**a) 1NF – First Normal Form:**

- No repeating groups
- Each cell has atomic (single) values

Example:
phone = "99999, 88888"  - wrong

Normalized:
Separate phone numbers into another table.

**b) 2NF – Second Normal Form:**

- Be in 1NF
- Remove partial dependency (applicable when composite key exists)

Example:
If table StudentCourse(student_id, course_id, student_name)
→ student_name depends only on student_id
→ Move student details to Student table.

**c) 3NF – Third Normal Form:**

- Be in 2NF
- Remove transitive dependency

Example:
If Student table has (student_id, course_id, course_name)
course_name depends on course_id, not student_id
→ Move course_name to Course table.

## ACID Properties:

Ensures safe, reliable transactions.

**A – Atomicity**

All steps succeed or none.

Example: Money transfer: debit + credit.

**C – Consistency**

Database moves from one valid state to another.

**I – Isolation**

Parallel transactions do not affect each other.

**D – Durability**

After commit, data is permanent even after crash.

## What is SQL:

SQL (Structured Query Language) is used to interact with RDBMS for creating, modifying, retrieving and managing data.

## SQL Commands:

SQL Commands are the set of instructions used to communicate with a database to create tables, store data, modify data, retrieve data, control user permissions, and manage transactions.

## DDL – Data Definition Language

DDL commands are used to **define and change the structure** of database objects like tables.

### CREATE

Used to create new database objects such as tables, databases, views, etc.

### ALTER

Used to modify the structure of an existing table—such as adding, deleting, or modifying columns.

### DROP

Used to permanently delete an entire database object (like a table or database).

### TRUNCATE

Used to remove all data from a table while keeping the table structure intact.

## DML – Data Manipulation Language

DML commands are used to **insert, update, or delete** data inside tables.

### INSERT

Used to add new records (rows) into a table.

### UPDATE

Used to modify existing data in one or more rows.

### DELETE

Used to remove specific rows from a table.

## DQL – Data Query Language

DQL deals with **retrieving and querying data** from tables.

### SELECT

Used to fetch data from one or more tables based on given conditions.

## DCL – Data Control Language

DCL commands handle **permissions and access control** within the database.

### GRANT

Used to provide specific privileges to users.

### REVOKE

Used to remove previously assigned privileges from users.

## TCL – Transaction Control Language

TCL commands manage **transactions** that involve multiple SQL operations.

### COMMIT

Saves all changes made in the current transaction permanently.

### ROLLBACK

Reverts changes made in the current transaction back to the last saved state.

## SAVEPOINT

Creates a checkpoint within a transaction to roll back to a specific point if needed.